You can remove this dependency from the action and the test by putting the dependency back in the Project method, which is already tied to users, and adding in some judicious test doubles to block the user from being called.

The new test looks like this:

environment/02/spec/workflows/creates_project_spec.rb
```
it "adds users to the project" do
  project = Project.new
  user = double
  expect(project).to receive(:add_users).with([user])
  allow(Project).to receive(:new).and_return(project)
  creator = CreatesProject.new(name: "Project Runway", users: [user])
  creator.build
end
```

The test is started with four lines setting up test doubles. There's a common pattern here, where you create a blank object of a certain type—in this case, Project—then stub Project.new to return that instance. This allows you to set further stubs on the specific project instance being used, secure in the knowledge that the code being tested will be using the same instance. Without stubbing Project.new, the new instance created in the action code would not have the stub assigned to the project instance in this test.

There are at least two other ways to handle this situation. You could use the RSpec framework's allow_any_instance_of feature:

```
allow_any_instance_of(Project).to receive(:add_users).with([user])
```

This would save a line in the setup. I try to avoid allow_any_instance_of these days on the grounds that it's too big and broad a weapon for the job. I prefer being more specific about what I want the code to do.

The RSpec team recommends against using this feature. You could instead move the Project.new call to its own method inside the workflow object and stub that method, which I would be more likely to do.

For the preceding code to work, you need to move the project.users = call inside the Project class:

environment/02/app/models/project.rb
```
def add_users(users)
  self.users << users
end
```

And then update CreatesProject to use that new method: